

Article

Agent-Based Decentralized Manufacturing Execution System via Employment Network Collaboration

Moonsoo Shin 

Department of Industrial and Management Engineering, Hanbat National University, 125, Dongseo-daero, Yuseong-gu, Daejeon 34158, Republic of Korea; shinms@hanbat.ac.kr; Tel.: +82-42-821-1758

Abstract

High variability in multi-product manufacturing environments and rapidly changing customer demands make decentralized coordination of work-in-process (WIP) and production resources increasingly important. However, the intrinsic rigidity of conventional centralized and monolithic manufacturing execution systems (MESs) renders them unsuitable for such highly dynamic environments. To address this limitation, this study proposes an agent-based distributed, decentralized MES architecture. The manufacturing execution process is realized through collaboration among constituent agents based on an employment network (EmNet). Specifically, three types of agents are introduced: WIPAgents (representing WIPs), PAgents (representing processing resources), and MHAgents (representing material-handling resources). Collaboration among agents (e.g., collaborator discovery, partner selection, and data sharing/exchange) is facilitated by a data-space-based collaboration platform which was introduced in our prior work. To validate the proposed architecture, we built a digital-twin-based simulation testbed and conducted simulation experiments. The experimental results confirm the validity and operational feasibility of the proposed architecture.

Keywords: agent-based systems; manufacturing execution system (MES); decentralized manufacturing; employment network (EmNet); multi-agent coordination; data-space collaboration; resource allocation and partner selection; shop-floor scheduling

1. Introduction

Current manufacturing systems face dynamic environments characterized by high complexity, variability, and uncertainty. Conventional manufacturing execution systems (MESs) typically exhibit a centralized, monolithic structure that ensures predictability in stable environments; however, this architecture constrains agility, flexibility, and adaptability in volatile conditions, especially when the system is subject to disruptions [1–4]. As the complexity and scale of the environment increase, these systems become increasingly difficult to design and slow to reconfigure [5–7]. Even minor disruptions—such as unexpected equipment failures or urgent order changes—often require comprehensive software modifications, rendering single, large-scale systems inefficient in responding to rapid changes [5,8].

To overcome these limitations, agent-based MES architectures that decentralize control decision-making are attracting attention as an alternative [9,10]. Each agent makes decisions independently and, when necessary, negotiates with other agents to operate the overall manufacturing execution mechanism. Consequently, even when disruptions occur, the overall system can respond autonomously and quickly. Consequently, such



Academic Editor: Alexandre Carvalho

Received: 28 November 2025

Revised: 19 December 2025

Accepted: 26 December 2025

Published: 30 December 2025

Copyright: © 2025 by the author.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

agent-based MESs enhance agility and adaptability to change and facilitate system reconfiguration by decentralizing control [11,12]. Many studies have demonstrated that agent-based distributed MESs can respond to changes faster and more robustly than centralized MESs [13–17]. In summary, agent-based MESs offer a resilient and adaptable alternative to rigid centralized systems by leveraging distributed control and real-time reconfigurability [18–20]. Nevertheless, agent-based MESs still have significant limitations. While fully decentralized, heterogeneous control schemes are robust in rapidly changing environments, they often suffer from limited global coordination and optimization. According to Valckenaers [5], in purely non-hierarchical multi-agent systems (MASs), agents make myopic judgments and actions, rendering global system optimization virtually unattainable. Moreover, system-level predictability is limited [5]. In addition, current agent-based MESs face various problems, such as short-horizon decision-making, semantic heterogeneity, and communication overheads, which remain open research challenges [5,18].

To address these issues, the implementation of a dynamic organizing mechanism for agent-based MESs is necessary. Dynamic organizing refers to the flexible adjustment of the organizational structure and interaction patterns of agents in response to changing circumstances. Dynamic organizing allows the MES to balance local autonomy and global coherence as needed and reconfigure its decision-making structure in real time. Under nominal conditions, agents collaborate horizontally; when instability is detected, a temporary coordinator restores order, after which control reverts to the distributed agents [21]. This concept is implemented in a hybrid architecture that combines decentralized and semi-centralized control modes. Dynamic organizing mechanisms simultaneously enhance resilience and efficiency by enabling self-reorganization of production resources. Fundamentally, in distributed and collaborative production environments, no single control strategy can always be optimal, making adaptability crucial. Dynamic organizing also serves as a robust coordination mechanism for maintaining global consistency across the system without the need for permanent central control. In this context, the instability caused by reactive negotiation in distributed decision-making can be mitigated by establishing a clearly defined temporary hierarchy or explicit contract-based plans among agents [19,22]. Furthermore, semantic alignment based on a shared ontology is necessary so that agents can correctly understand each other's capabilities and requests during the reorganization process [6,23,24]. However, the challenge of effectively organizing distributed entities remains unresolved. Prior research has predominantly analyzed the final organizational structures such as a holarchy and the behaviors of individual components, rather than the formation process itself. Therefore, establishing a robust organizing principle is essential to provide a comprehensive solution to this problem. Furthermore, to the best of our knowledge, no study has addressed the detailed behaviors of constituent agents at a level applicable to real-world MES environments.

The primary objective of this study is to propose an agent-based distributed MES architecture. Specifically, it addresses three open problems: (1) how to organize distributed entities, (2) how to facilitate collaboration among them, and (3) how to implement interactions between execution-layer agents and control-layer executors at a practical level. To this end, the proposed MES builds upon the employment network (EmNet)-based organizational model established in our prior research [25]. The EmNet framework enables the dynamic organization of production resources through the recursive propagation of employment relationships throughout the entire vertical hierarchy. Furthermore, the communication and collaboration for this process are supported by a data-space-based collaboration platform, as presented in our previous work [26]. This study focuses on resource management and scheduling/dispatching—key functional areas defined in standard MES models. To demonstrate the validity and operability of the proposed manufacturing

execution architecture, a digital-twin-based simulation testbed is developed, and simulation studies are conducted using this platform. However, the specific decision models of individual agents and the technical details of the data-space platform are beyond the scope of this study.

The study is organized as follows: In Section 2, we introduce the relevant research trends. First, we review the literature on agent-based MESs. Then, we briefly introduce the EmNet-based collaboration model and data-space-based collaboration platform upon which this study is based. In Section 3, we present the agent-based distributed MES architecture proposed in this study. Then, in Section 4, we present a digital-twin-based simulation testbed to verify the proposed architecture. In Section 5, we verify the validity and operability of the proposed architecture through simulations. Finally, we conclude the study and discuss our future research in Section 6.

2. Related Works

2.1. Agent-Based MES

Agent-based MESs are developed with the aim of enhancing shop-floor autonomy and flexibility [27]. Representative architectures include the holonic manufacturing system (HMS) [28–30], the fractal manufacturing system (FrMS) [31–33], and the bionic manufacturing system [34]. These establish a quasi-hierarchical structure based on production units (e.g., holons, fractals, cells) with autonomy and self-organization capabilities. For example, the PROSA (Product-Resource-Order-Staff Architecture), proposed by Van Brussel et al. [28], is a hybrid architecture that simultaneously implements the advantages of hierarchical and heterarchical control based on the cooperative structure of holarchy among the product holon, the resource holon, and the order holon [30]. The PROSA is considered a landmark work in the field of HMSs, and many subsequent studies have adopted and expanded upon its ideas. In particular, the ADACOR (ADaptive holonic COntrol aRchitecture) is an architecture that develops upon the concept of the PROSA and introduces an adaptive hybrid control mechanism [22]. In the ADACOR, each holon operates a cooperative system at the same level, and a higher-level supervisor holon performs global coordination functions as needed. This partial hierarchical structure can alleviate the difficulty of achieving global optimization—a drawback of fully distributed architectures—and enhance resilience [21]. The fractal factory, first proposed by Warnecke [31], defines the basic organizational unit of a production system as a self-similar agent with autonomy and dynamic reconfiguration capabilities, or “fractals”, and aims to enhance resilience and flexibility in resource management and scheduling [35]. This framework has inspired numerous agent-based MES implementations, in which intelligent resource agents and product agents collaboratively perform scheduling and dispatching tasks on the shop floor.

The coordination mechanisms of agent-based MESs typically rely on decentralized architectures and on negotiation and cooperation among agents, as opposed to top-down control. A representative negotiation-based coordination mechanism is the Contract Net Protocol (CNP), first proposed by Smith [36]. Each work agent (or task agent) announces a bid, and resource agents participate in the bid, considering their availability. Each bid is evaluated based on the task’s requirements and preferences, and the optimal proposal is selected and a contract is awarded. This iterative process creates a matching process between individual tasks and resources, similar to market trading. This approach is well suited for real-time rescheduling and can respond agilely to changes in production plans or unexpected disruptions (such as equipment failures or urgent orders). For example, product agents and resource agents can continuously negotiate and reschedule work start times based on environmental changes. Furthermore, it is widely used to implement high-quality distributed scheduling through extensions such as multi-round bidding and

utility-based awards [37]. Furthermore, various distributed collaboration methods, such as the blackboard system and cooperative distributed optimization, can be applied. Agents negotiate constraints, start times, priorities, and other factors to pursue local optima, thereby enhancing the resilience and fault tolerance of the overall system [38]. This provides the advantage of enhanced robustness against disturbances at the expense of some global optimality [39].

Agent-based MESs have been applied across various industries. They are particularly effective in complex and dynamic environments, such as flexible manufacturing systems (FMSs) and job shops. For example, Hussain and Ali [40] proposed a dynamic scheduling system in which machine agents and part agents negotiate schedules to flexibly respond to machine failures and workload variations. Arai et al. [41] implemented a holonic assembly system capable of plug-and-produce, where autonomous robots and product agents collaborate within an assembly cell. In highly variable and mixed production environments—such as those that are typical in the semiconductor, electronics, and automotive industries—this approach has been demonstrated to have superior reconfigurability and adaptability compared to conventional static MESs [27]. Some studies have reported that real-time negotiation between product and resource agents improves on-site responsiveness to events such as rush orders and equipment failures [9,10]. However, these existing studies typically focus on collaboration among agents within pre-established organizational structures, rather than addressing the dynamic organizing process itself.

2.2. EmNet-Based Collaboration

This study adopts a dynamic reconfiguration framework based on EmNet. In this section, we introduce the concept of EmNet and investigate the relevant research works. EmNet is a dynamic organizational model of production resources built on temporary employment contracts among those resources [25]. Within EmNet, each resource simultaneously plays the roles of employer and employee, forming relationships through decentralized, autonomous contract negotiations. EmNet flexibly establishes and dismantles hierarchical structures through temporary employment contracts between production resource agents, thereby ensuring both global consistency and local autonomy in the absence of central control. For example, a higher-level agent hires lower-level resource agents and assigns tasks, and these agents rehire additional resources as needed. The organizing mechanism recurses across levels (via employment chains), enabling real-time reconfiguration. Because this structure establishes contractual relationships on a job-by-job basis, the scope of negotiation is naturally limited, reducing communication overheads and enhancing real-time performance. EmNet facilitates integration with legacy MESs through an employment contract interface that is similar to those used in extant organizational structures. Its self-organizing mechanism establishes a fractal organization and supports real-time reorganization in response to changing situations, such as breakdown response, resource expansion, and process transitions. Figure 1 illustrates the resulting fractal organization, in which employment relationships recur in a self-similar manner across hierarchical levels.

Building on this fractal concept, our previous works [42,43] introduced the Autonomous and Intelligent Resource unit (AIR-unit) and established a contract-based operation model. Subsequently, this framework was extended in [25] to form a dynamic organizational model that reflects both the hierarchical structure of production resources and the collective definition of production activities. In this model, distributed and decentralized resources are organized through a dynamic process for establishing employment contracts across abstraction levels, and capacity planning, production planning, and scheduling are performed cooperatively by resources modeled as AIR-units. Although collaboration primarily emphasizes vertical interactions between higher- and lower-level

resources, it naturally supports horizontal coordination within a layer and can extend to inter-enterprise collaboration along a supply chain, since all resources are modeled as autonomous, decentralized decision makers [25,33,43].

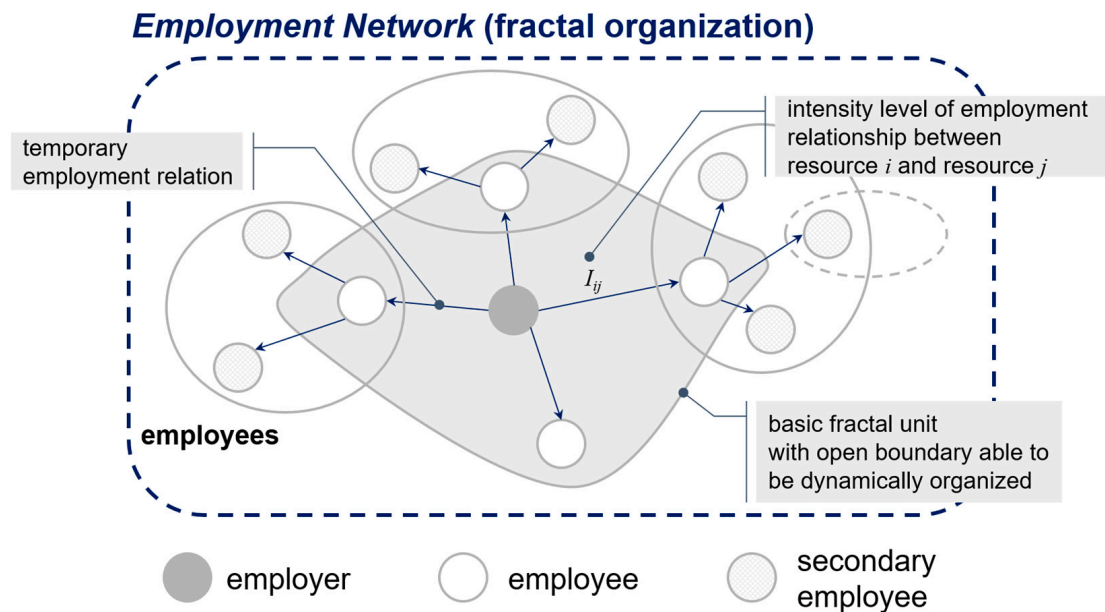


Figure 1. EmNet-based fractal organization (after [25]).

Prior research [26] has formalized a collaboration model among autonomous, distributed resources in a supply chain; here, an EmNet was established via temporary contracts between Supplier and Customer agents. Each production resource is represented as an AIR-unit that makes local decisions while contributing to global coordination through two interrelated processes: the relation-forming process (configuring who collaborates with whom) and the task-allocating process (deciding what is executed, when, and by whom). By contrast, this study proposes an operational framework for a distributed, collaborative agent-based MES architecture by adapting the collaboration mechanism inherent in EmNet. Production resources within a manufacturing system collaborate with one another, assuming the dual role of employer and employee to carry out manufacturing execution tasks. Further details are provided in Section 3.

2.3. Data-Space-Based Collaboration Platform

A data-space is a decentralized, federated infrastructure that enables trustworthy data sharing while preserving data sovereignty. The International Data Spaces Association (IDSA) defines the key building blocks of a data-space as follows: (1) a data platform that facilitates exchange and interoperability; (2) data marketplaces where participants publish, discover and transact capabilities and services; and (3) functional services for sovereignty, including identity management, access control, and usage policy enforcement [44]. Gaia-X federation services—such as identity and trust, federated catalogue, sovereign data exchange, and compliance—concretize these functions to operationalize interoperability across distributed, heterogeneous participants [45].

While initiatives such as Catena-X emphasize supplier search from the demand side to match capabilities in discrete manufacturing contexts [46,47], prior research [26] highlights the fact that supplier-driven task allocation becomes crucial when one is prioritizing and distributing production capacity; this is particularly the case in asset-intensive environments. To address this requirement, a data-space-native collaboration platform was proposed in [26], featuring the architecture shown in Figure 2. This platform comprises

three functional components: (1) a data-gathering interface that ingests data from distributed sources and optionally employs blockchain or other ledgers to ensure auditability and data sovereignty; (2) a data-sharing platform that applies semantic mapping (via ontology or data dictionary) to support interoperability and policy compliance; and (3) a data marketplace that catalogs and transacts manufacturing capabilities and capacities across participants. From these, the key architectural requirements derived include decentralization, interoperability, openness to dynamic membership, and scalability within and across organizational layers.

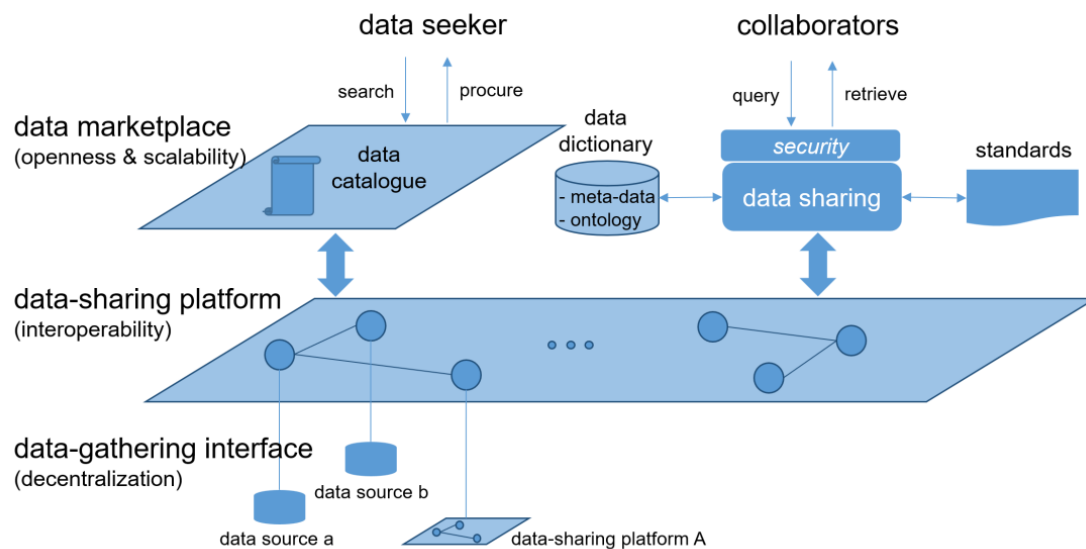


Figure 2. Conceptual structure of a data-space-based collaboration platform (after [26]).

The platform provides the underlying infrastructure for embedding EmNet’s agent protocols within a secure, policy-compliant ecosystem. It supports both supplier-search and demand-search collaboration patterns and enables controlled, semantically aligned information sharing among autonomous agents. Beyond enabling inter-organizational data exchange, it facilitates agent-based collaboration for task allocation and production capacity distribution. A relation-forming phase allows agents to identify and contract with potential collaborators, while a task-allocating phase governs the distribution of workloads under employment contracts that specify capabilities, volume, time, and cost. Each entity is modeled as an AIR-unit that acts as both supplier and customer. By combining semantic interoperability with decentralized contract negotiation, the platform supports coordination without full information disclosure. Simulation experiments have shown that this model achieves performance comparable to centralized optimization while exceeding the responsiveness of conventional distributed methods [26].

By formally embedding agent-based manufacturing execution within a data-space architecture, this approach creates a unified framework that links supply-chain-level autonomy with execution-level responsiveness. Its generality makes it suitable for both continuous-process environments and dynamic, multi-party supply networks where decentralized coordination and secure data sharing are paramount. However, the specific technical implementation of the data-space platform is not within the scope of this study.

3. EmNet-Based Manufacturing Execution System

3.1. Conceptual Architecture

This section proposes a distributed, collaborative MES architecture based on the EmNet-based collaboration model. Specifically, we model the entities responsible for

manufacturing execution decisions as distinct types of agent objects and describe the manufacturing execution process that is implemented through their collaboration.

3.1.1. Constituent Agents

Three types of agents play a key role in decision-making for EmNet-based manufacturing execution: (1) WIPAgents, representing work-in-process; (2) PAgents, for processing resources; (3) MHAgents, for material-handling resources.

The WIPAgent represents a work order and participates in all decision-making processes necessary to complete that order. Each work order is defined at an abstraction level appropriate to the hierarchical structure of production resources. The inter-factory level handles high-level work orders (i.e., production orders) executed by each factory unit, while the shop-floor level handles work orders at the task level for each subordinate piece of production equipment. Essentially, each production order assigned to a factory unit and each WIP item on the shop floor is represented and managed by an individual WIPAgent.

The PAgent participates in decision-making on behalf of individual production resources, particularly processing equipment. It accepts and processes unit tasks based on the production capabilities and the available production capacity of the processing equipment. PAgents can also be defined in various hierarchical forms, depending on the level of abstraction of the production resources. At the inter-factory level, it represents a single factory, while at the inter-cell level, it represents each cell, comprised of various production equipment. A PAgent establishes an EmNet by entering into an employment contract with production resources in adjacent hierarchical layers.

The MHAgent represents individual pieces of material-handling equipment. Each material-handling equipment is responsible for transporting production resources at various levels, as defined by the hierarchical structure. This study does not consider the hierarchical structure of the material-handling equipment.

Figure 3 illustrates the operational workflow of the proposed EmNet-based MES. Upon the creation of a work order, a dedicated WIPAgent is instantiated. Subsequently, the WIPAgent secures the necessary manufacturing resources to process the order (or its associated WIP). This process entails two key phases: (1) relation-forming, which involves selecting resources with the required capability and capacity; and (2) task-allocating, which assigns specific unit tasks to these resources at the appropriate times. These operations are executed through interactions with PAgents and MHAgents, facilitated by the PlanMgr, Dispatcher, and Router sub-modules embedded within the WIPAgent.

In the relation-forming phase, the WIPAgent utilizes its PlanMgr module to delegate the order to a PAgent. Acting as an “employee”, the PAgent accepts the job and may subsequently employ other PAgents to handle sub-jobs through negotiation. Upon the completion of these resource assignments, a comprehensive production plan is finalized, and the EmNet is established. In the subsequent task-allocating phase, the WIPAgent uses its Dispatcher module to sequentially assign tasks to the employee agents within the EmNet, in response to load/unload requests from the shop floor. This task-allocating process is repeated until the given work order is completed. Simultaneously, the WIPAgent employs its Router module to negotiate with MHAgents to generate the necessary transport routes. Specific interactions between agents are described in detail in the following Section 3.2.

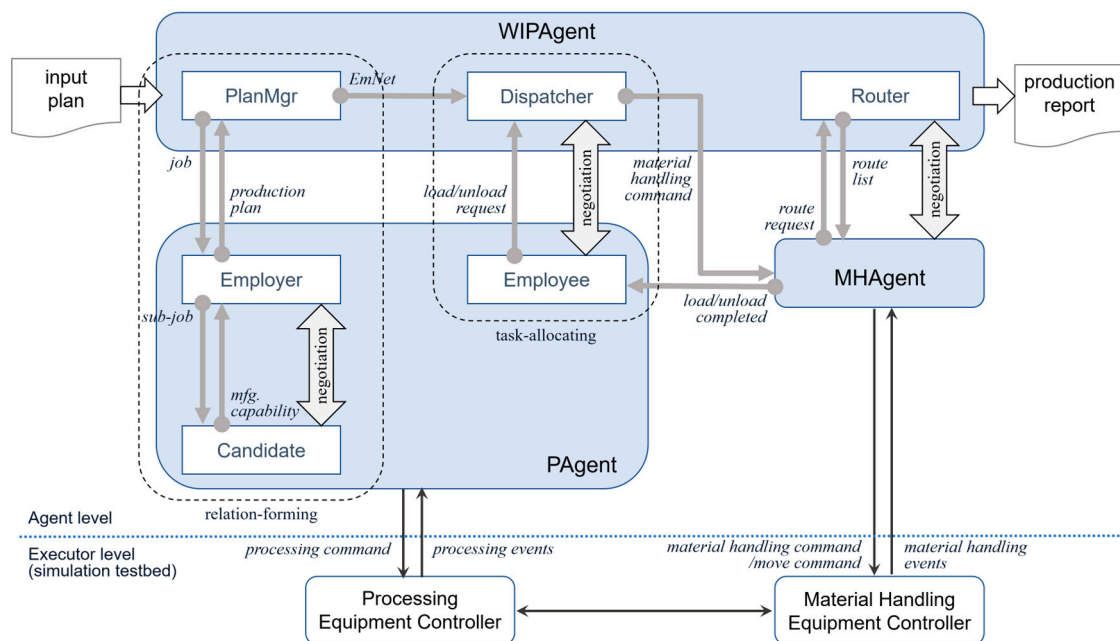


Figure 3. Operational workflow of the EmNet-based MES.

3.1.2. Manufacturing Execution Functions

The EmNet-based MES proposed in this study implements manufacturing execution functions through the interactions that occur between the WIPAgent, PAgent, and MHAgent, as presented previously. Specifically, this study focuses on the three key functions of a typical MES: (1) resource management, (2) scheduling and dispatching, and (3) tracking and genealogy.

The resource-management function can be viewed from two perspectives. One is the resource allocation (cf. MESA functions [48]) and resource management (cf. ISA-95 [49]) functions of traditional MES models, which refer to the management of individual production resources' availability, capability, and utilization rates (see Table 1). The other is dynamic resource allocation, as linked to production planning; this is a differentiating feature of the EmNet-based MES proposed in this study. During EmNet establishment and operation, the relation-forming mechanism dynamically reallocates production resources to fulfill given work orders and readjusts the production plan. This serves as a key element in enhancing the responsiveness and reconfigurability of the overall manufacturing system.

The scheduling and dispatching functions are implemented through the task-allocating process, as proposed through the EmNet-based collaboration model. Detailed tasks are allocated based on the employment relationships specified by EmNet, and this process generates concrete task schedules and allocations. The specific details of the relation-forming and task-allocating processes are discussed in Section 3.2.

Finally, the tracking and genealogy function is implemented based on the events generated according to the behavior model embedded in each agent. This allows for the collection and management of all production history data that require tracking, from the creation of WIP to the allocation of production resources and the start and end points of tasks. This extends to data collection and performance analysis capabilities. Event-based tracking and history management processes are covered in detail in Section 4.

Table 1. Functional alignment of MES functions, as defined by MESA International [48] and ISA-95 [49].

Unified Category	ISA-95 Function(s)	MESA Function(s)	Summary of Role
Production Scheduling	Detailed Production Scheduling	Operations/ Detailed Scheduling	Translating high-level production plans into executable schedules by reflecting real-time resource constraints.
Dispatching and Execution Control	Production Dispatching	Dispatching Production Units; Process Management	Coordinating the real-time release, sequencing, and monitoring of work orders and process operations.
Resource and Labor Management	Resource Management	Resource Allocation and Status; Labor Management	Managing the availability, capability, and utilization of resources such as equipment, tools, materials, and personnel.
Quality and Compliance Management	Quality Operations Management	Quality Management; Document Control	Maintaining quality and compliance through inspection, data collection, and document management.
Maintenance and Asset Management	Maintenance Operations Management	Maintenance Management	Managing maintenance schedules (preventive/corrective) and monitoring asset health to maximize system availability.
Material/WIP Tracking and Genealogy	Tracking and Genealogy; Inventory Operations Management	Product Tracking and Genealogy	Tracking materials, components, and products end-to-end to ensure WIP visibility and maintain product genealogy.
Data Collection and Performance Analysis	Data Collection; Performance Analysis	Data Collection/Acquisition; Performance Analysis	Acquiring real-time data to calculate KPIs (e.g., OEE, yield) and enabling data-driven continuous improvement.

3.2. Collaboration Mechanism

The collaboration process unfolds in two phases—relation-forming (organizational planning) and task-allocating (execution scheduling). These processes are implemented upon a data-space infrastructure that ensures sovereign and interoperable data exchange. The overall collaboration process is performed via employment contracts between manufacturing resources.

3.2.1. Employment Contract

Formally, each resource-side agent (i.e., PAgent and MHAgent) can be seen as an AIR-unit, as presented in Section 2.2:

$$\text{AIR-unit} = (C, J, T), \quad (1)$$

where C is the set of available capabilities, J is the set of jobs (employment contracts), and T is the set of allocated tasks. Each agent evaluates decisions using a multi-objective fuzzy goal function as follows [25]:

$$GA = \sum_i w_i \mu_i(x_i), \quad \text{where } \sum_i w_i = 1 \quad (2)$$

x_i denotes the sub-goal performance (e.g., tardiness, cost, energy), $\mu_i(\cdot)$ is its fuzzy membership function, and w_i is its weight factor; GA represents the level of overall goal achievement. Figure 4 illustrates the conceptual model of an employment contract between AIR-

units using a UML (Unified Modeling Language) class diagram. In this context, individual AIR-units act as either employers or employees.

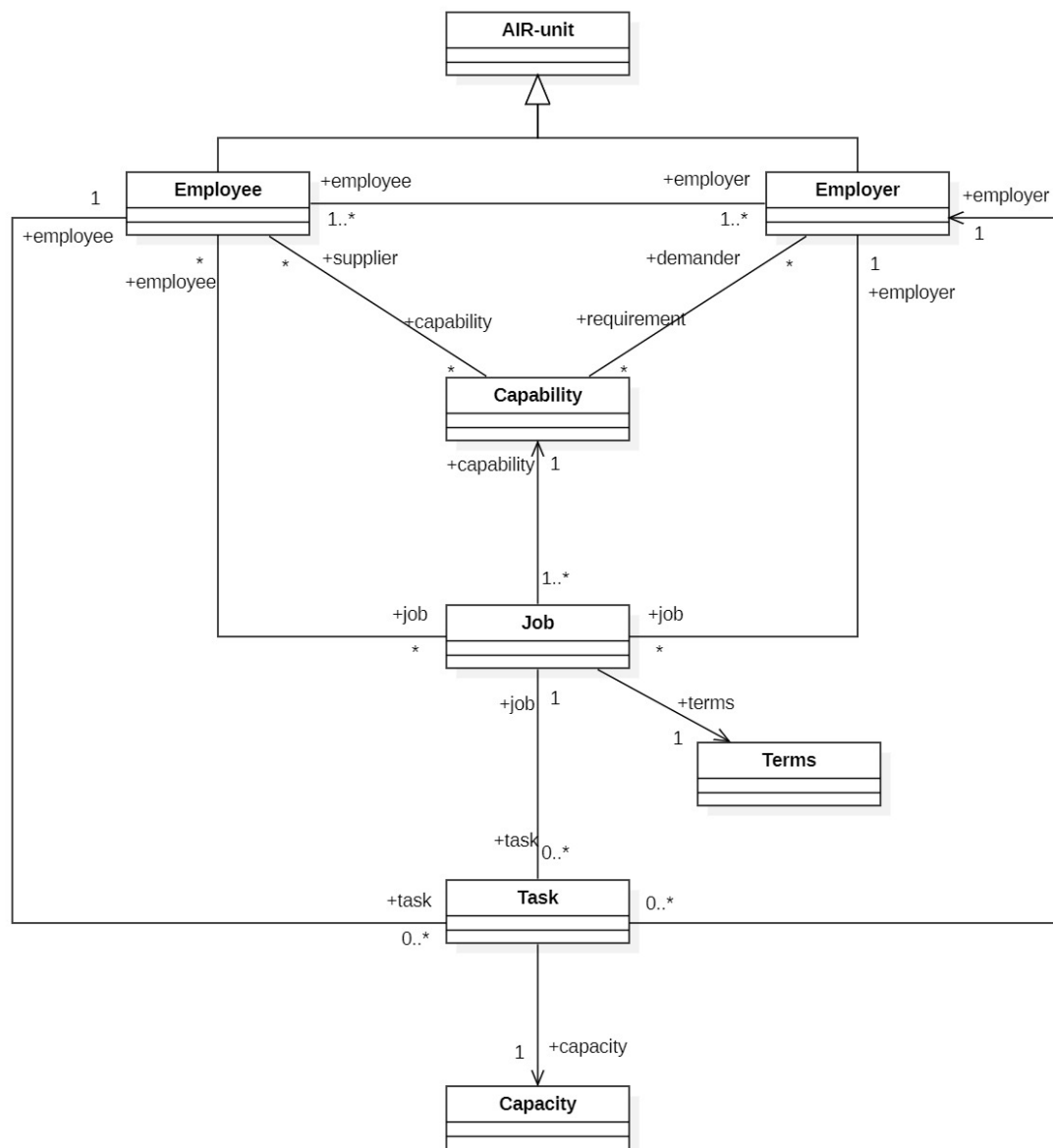


Figure 4. Conceptual model of an employment contract between AIR-units (adapted from [26]).

3.2.2. Relation-Forming (Planning)

Relation-forming establishes employment contracts from the employer's side. A *job* encapsulates an employment relation to provide a capability over a term, which is defined as follows:

$$job = (cap, employee, terms, T), \quad (3)$$

where *cap* denotes the capability, *employee* refers to the agent assigned to the job, *terms* represents the contract conditions (e.g., time horizon, cost/reward, required service levels), and *T* is a set of associated tasks, which may be empty. The employer-side agents explore the capabilities of the candidate employees (via a marketplace), request terms, evaluate offers and select the best contract as follows:

$$job^* = \operatorname{argmax}_{job \in BID} GA^{employer}(job), \quad (4)$$

which $GA^{employer}(job)$ represents the level of preference associated with a given *job* from the employer's perspective and BID^J denotes a set of jobs proposed by candidate employees. The relation-forming phase results in an employment network that temporarily binds parties and enables dynamic reorganization.

The relation-forming process is initiated when a WIPAgent—acting as a plan manager (named PlanMgr) responsible for production planning and resource assignment—delegates a production order (i.e., processing a WIP) to a PAgent. Upon receiving this order, the PAgent assumes the role of an employer, decomposing the task into necessary sub-jobs. These sub-jobs are subsequently allocated to other PAgents acting as employees. To facilitate this coordination, all PAgents register their executable capabilities in a matchmaking registry designated as the MarketPlace. This module is responsible for advertising and brokering manufacturing services, thereby enabling employer PAgents to discover candidate employees from the pool. Figure 5 illustrates the interactions among the major stakeholders involved in the relation-forming process; Table 2 presents detailed descriptions of their respective activities.

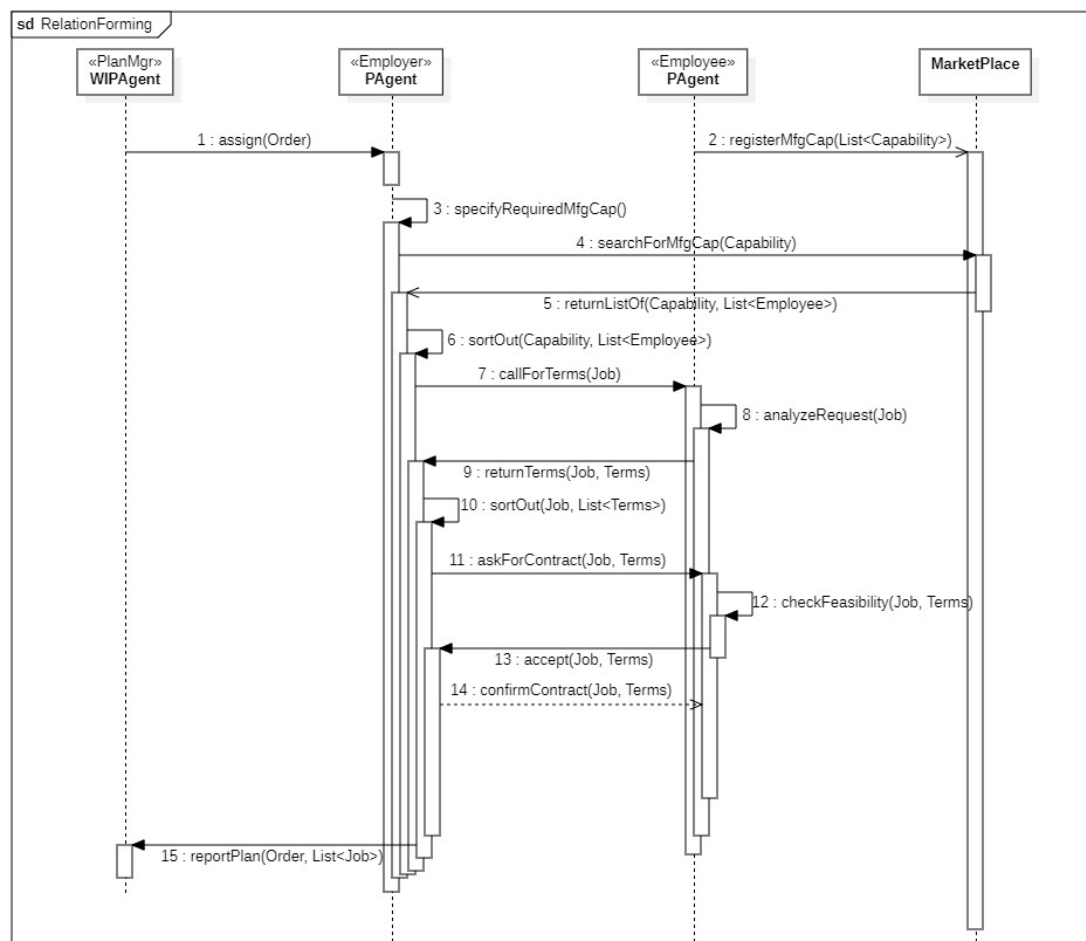


Figure 5. Relation-forming process through collaboration among PlanMgr, Employer, Employee, and MarketPlace objects (adapted from [26]).

Table 2. Descriptions of activities of stakeholders in a relation-forming process.

Stakeholder	Function	Description
<<PlanMgr>> WIPAgent	assign (Order)	Assigning a production order to an AIR-unit responsible for acting as the employer.
	specifyRequiredMfgCap ()	Specifying the list of required manufacturing capabilities.
	searchForMfgCap (Capability)	Searching for candidates capable of providing the given capability.
<<Employer>> PAgent	sortOut (Capability, List <Employee>)	Sorting out candidates for the employee to provide the given capability.
	callForTerms (Job)	Calling for contract terms for the given job.
	sortOut (Job, List <Terms>)	Sorting the list of the terms provided by the candidate employees.
	askForContract (Job, Terms)	Requesting a contract for the given job.
	confirmContract (Job, Terms)	Confirming a contract for the job according to the terms presented.
<<Candidate>> PAgent	reportPlan (Order, List <Job>)	Reporting on the list of jobs associated with the production order.
	registerMfgCap (List <Capability>)	Registering the list of manufacturing capabilities.
	analyzeRequest (Job)	Analyzing the requested job and specifying the contract terms for the job.
	returnTerms (Job, Terms)	Returning the contract terms for the job.
	checkFeasibility (Job, Terms)	Checking the feasibility of the terms for the job.
	accept (Job, Terms)	Accepting the terms for the job.
MarketPlace	deny (Job, Terms)	Denying the terms for the job.
	returnListOf (Capability, List <Employee>)	Returning the list of candidate employees capable of providing the given manufacturing capability.

3.2.3. Task-Allocating (Detailed Scheduling/Dispatching)

A *task* is a capacity commitment attached to a *job* as follows:

$$task = (job, employee, capacity, terms), \quad (5)$$

where *capacity* denotes the quantity of required workload (represented as lot quantity, batch membership, and a start window) and *terms* denotes contract terms (including time horizon, cost/reward). Tasks required for completing the assigned job are allocated by an employer-side agent to employees defined within the scope of contracted employment relationships (i.e., jobs). Employee-side agents bid on the terms associated with the tasks from their own perspective. Finally, the employer-side agent selects among feasible task sets to maximize its objectives:

$$task^* = \operatorname{argmax}_{task \in BID^T} GA^{employer}(task) \quad (6)$$

where $GA^{employer}(task)$ represents the level of preference associated with a given *task* from the employer's perspective, and BID^T denotes the set of tasks proposed by candidate employees.

The task-allocating process begins when a WIPAgent, acting as a dispatcher, issues a call for terms to the employee-side PAgents associated with a task; this occurs after the WIPAgent has analyzed and prioritized the list of tasks to be processed. Figure 6 illustrates

the interactions among the major stakeholders involved in the task-allocating process; Table 3 presents detailed descriptions of their respective activities.

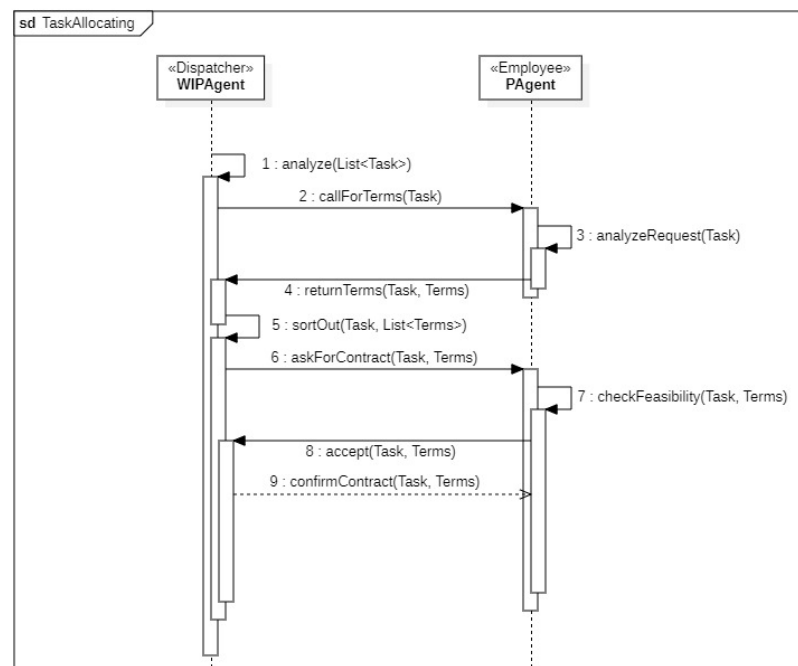


Figure 6. Collaboration process for task-allocating by Dispatcher and Employee objects.

Table 3. Descriptions of the activities of stakeholders in a task-allocating process.

Stakeholder	Function	Description
<<Dispatcher>> WIPAgent	analyze (List <Task>)	Analyzing the tasks associated with a job and identifying the employees assigned to it.
	callForTerms (Task)	Calling for terms associated with the given task.
	sortOut (Task, List <Terms>)	Evaluating a list of terms provided by its employees and selecting the task with the most favorable terms.
	askForContracts (Task, Terms)	Asking for a task contract under the specified terms.
	confirmContract (Task, Terms)	Confirming the task allocation under the given terms.
<<Employee>> PAgent	analyzeRequest (Task)	Analyzing a given task and generates its associated terms.
	returnTerms (Task, Terms)	Returning a task along with its associated terms.
	checkFeasibility (Task, Terms)	Determining the feasibility of executing the task under the provided terms.
	accept (Task, Terms)	Accepting the proposed task in accordance with the given terms.
	deny (Task, Terms)	Denying the proposed task in accordance with the given terms.

3.3. Relation to Previous Work

Building upon the EmNet-based collaboration model and the data-space-based platform established in our previous works [25,26], this study aims to develop an agent-based distributed MES. In this section, we clarify the distinct contributions of this study compared to the previous work across four key aspects: (1) conceptual level, (2) architectural extension, (3) integration with the execution layer, and (4) new experimental insights.

While the previous study addressed a macro-level model focusing on inter-enterprise supply chain management, this study targets a micro-level model centered on the operation of individual production resources within a plant (intra-plant). Specifically, the previous work characterized collaboration as a horizontal relationship driven by capacity trading and “supply contracts” between suppliers and customers. In contrast, the collaboration mechanism proposed in this study operates as a vertical relationship within a hierarchical structure, where upper-level resources dynamically hire lower-level resources. Notably, by focusing on an intra-plant MES that governs shop-floor resources, this study encompasses not only logical task allocation but also the physical execution of manufacturing processes. Ultimately, it is anticipated that it will be possible to integrate inter-enterprise supply chain management and intra-plant manufacturing execution into a single, unified EmNet-based collaboration model.

In this study, the roles of “customer” and “supplier” from our previous work are redefined as “employer” and “employee”, respectively, to better represent the vertical collaborative relationships among hierarchical production resources. Furthermore, to model the manufacturing execution process on the shop floor, we introduce three specific agent types: (1) WIPAgent, (2) PAgent, and (3) MHAgent. The introduction of the MHAgent is particularly significant as it incorporates the physical transport constraints of WIP. Given that WIP transportation is a major operational constraint alongside processing, the inclusion of the MHAgent is a critical element for the successful implementation of an intra-plant MES.

This study integrates physical constraints at the executor level into agent-level decision-making. This integration is realized through a digital-twin-based simulation testbed, which links the agents to physical executors and is detailed in Section 4. Consequently, specific shop floor constraints—such as buffer capacities, loading/unloading events, and physical transit times—are explicitly reflected in the decision-making process. In contrast, our previous work relied solely on purely logical simulations, considering only estimated processing times during the capacity trading and task allocation process.

While the previous study focused on evaluating task allocation efficiency at the supply chain management level—demonstrating that EmNet-based decentralized collaboration is comparable to centralized decision-making—this study shifts the focus to dynamic resilience. We validate system performance under various disturbance scenarios, such as machine breakdowns, rush orders, and processing delays. Given that the ultimate goal of adopting an agent-based distributed architecture is to enhance agility, flexibility, and adaptability, performance evaluation within a dynamic environment is crucial. The detailed experimental design and results are presented in Section 5.

4. Digital-Twin-Based Simulation Testbed

This section presents the EmNet-based MES and a simulation testbed as a prototype implementation of the architecture proposed in Section 3. The proposed prototype realizes the EmNet-based manufacturing execution processes and verifies its operability using an agent-based digital-twin system. In this context, the digital-twin virtualizes various shop-floor-level executors (e.g., processing equipment, material-handling equipment) as agent objects and simulates the collaboration between MES agents and these executors. The MES agents act as autonomous decision-making entities during the collaboration process with shop-floor-level executors. Because the decision-making algorithms themselves are beyond the scope of this study, this section focuses on verifying the operability of the proposed mechanism by validating the interactions between MES agents and executors within the digital-twin rather than quantitatively evaluating performance.

4.1. System Configuration

The EmNet-based MES proposed in this study operates based on decision-making through collaboration among agents at the MES level. For instance, once task allocation is determined through negotiation between the WIPAgent and the PAgent, the corresponding work instruction is delivered to the shop floor, and the MES receives feedback on its execution status and results. At this point, various executors located at the shop-floor layer interact as counterparts to the MES agents. Figure 7 illustrates the structure of this system. The decision-making process within the MES is realized through interactions among agent objects equipped with autonomous decision-making capabilities, whereas the shop floor consists of executors that respond to the decisions made by MES agents. The simulation testbed serves as a digital-twin of the physical shop floor, emulating its responses to MES execution control. Communication between agents and shop-floor executors is facilitated via arbitrary communication interfaces (e.g., OPC-UA).

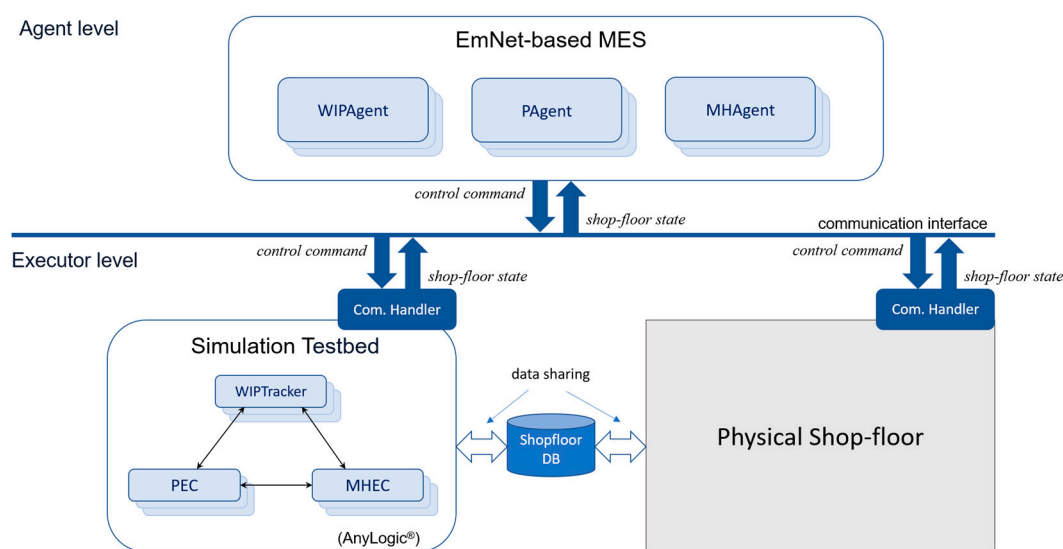


Figure 7. System configuration of the EmNet-based MES integrated with a simulation testbed.

The shop-floor executors are categorized into the following three types:

- **WIPTracker:** It simulates the behavior of WIP on the shop floor and reports tracking information of individual WIPs to their corresponding WIPAgents. Each WIPTracker is linked one-to-one with a WIPAgent in the MES.
- **PEC (processing equipment controller):** It simulates the behavior of processing workstations and reports status changes of individual workstations to the MES. Each PEC is typically linked one-to-one with a PAgent, receives control commands from the associated PAgent and reports on the execution results. It also interacts with WIPTrackers to induce changes in WIP processing status.
- **MHEC (material-handling equipment controller):** It simulates the behavior of material-handling equipment and reports status changes to the MES. Each MHEC is typically linked in a one-to-one manner with an MHAgent, receives control commands from the associated MHAgent and reports on the execution results. It specifically handles WIP transportation, thereby inducing changes in WIP status.

4.2. Reference Model of the Physical Shop Floor

In this section, a reference model of the target manufacturing system for MES implementation is presented. The model separates the processing flow from the material-handling flow, representing each as a discrete event model. Furthermore, the interaction

mechanism between MES agents and shop-floor executors is defined via event and message specifications. A prototype implementation based on this framework is detailed in Section 4.3.

4.2.1. Processing Model

The behavior of processing workstations can be defined by the workflow of each WIP item within the workstations. The event-driven flow of WIP in an individual workstation is structured as follows (see Figure 8). Each workstation consists of a processing unit and input/output buffers. When a WIP is loaded into the input buffer, an LDCom (load completed) event is triggered. Once the WIP enters the processing unit, a TkIn (track in) event occurs; simultaneously, an LDReq (load requested) event is generated to request the loading of the next WIP. The STA_EX event is triggered at the start of the process, and the END_EX event is invoked upon its completion. Upon completion of the processing, a TkOut (track out) event is triggered, followed by a ULDRReq (unload requested) event to initiate unloading to the output buffer. Finally, when the WIP is unloaded, a ULDCom (unload completed) event is triggered.

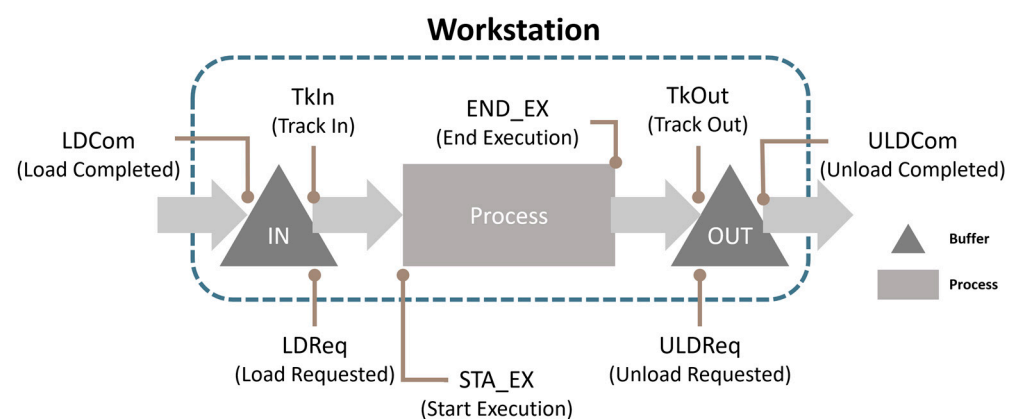


Figure 8. Reference model for the processing workflow.

The MES developed in this study manages unit operations based on this cycle, which also serves as the fundamental structure for WIP tracking. Table 4 details the specification of the events involved in this workflow.

Table 4. Discrete event model for the processing workflow.

Event		Description
LDCom	Triggering condition	TkIn or WakeUp occurs, provided that the inBuffer has sufficient loading capacity: - <code>isReadyForLDRequest(workstation) == true</code> - <code>&& inBuffer.getCapacity() > inBuffer.getWIPQty()</code>
	Action	Find a next WIP: - <code>wip = findWIPForLoadTo(workstation)</code> - <code>if(wip != null)</code> waiting for ULDCom on the source location of the WIP (i.e., <i>wip</i>) waiting for LDCom on the target location of the WIP (i.e., <i>wip</i>)
LDCom	Triggering condition	A WIP arrives at the inBuffer once either the WIP is determined by an LDReq or its destination is determined by a ULDRReq.
	Action	TkIn is triggered, provided that the process start conditions are fulfilled.

Table 4. Cont.

	Event	Description
TkIn	Triggering condition	LDCom occurs, provided that the process start conditions are fulfilled. - findWIPForTkIn(<i>workstation</i>) != null
	Action	The WIP is removed from the inBuffer. STA_EX is triggered.
STA_EX	Triggering condition	TkIn occurs.
	Action	Processing is initiated. END_EX is triggered after the processing time.
END_EX	Triggering condition	After STA_EX occurs and the processing time has elapsed.
	Action	Processing is completed. TkOut is triggered.
TkOut	Triggering condition	END_EX occurs.
	Action	ULDReq is triggered, provided that the transfer conditions are met. - findWIPForUnloadFrom(<i>workstation</i>) != true
ULDReq	Triggering condition	TkOut or WakeUp occurs, provided that the transfer conditions are met. - findWIPForUnloadFrom(<i>workstation</i>) != true
	Action	Find the next destination of the WIP. - <i>dest</i> = findWorkStationForULDReq(<i>workstation</i> , <i>wip</i>) - if(<i>dest</i> != null) waiting for ULDCom on the current location of the WIP (i.e., <i>wip</i>) waiting for LDCom on <i>dest</i>
ULDCom	Triggering condition	A WIP leaves the outBuffer once either the WIP is determined by an LDReq or its destination is determined by a ULDReq.
	Action	The transfer process is initiated.
WakeUp	Triggering condition	The event periodically Periodically occurs at a predefined frequency to check the trigger conditions of each event type (e.g., LDReq, ULDReq).
	Action	LDReq and ULDReq are triggered, provided that their respective conditions are met.

4.2.2. Material-Handling Model

The material-handling process is modeled independently of the processing operations at individual workstations. In this study, the material-handling model focuses solely on the transfer of WIP items between buffers, as illustrated in Figure 9. Specifically, the destination of a transfer is restricted to a buffer; thus, transfers to processing units are represented as transfers to their respective input buffers.

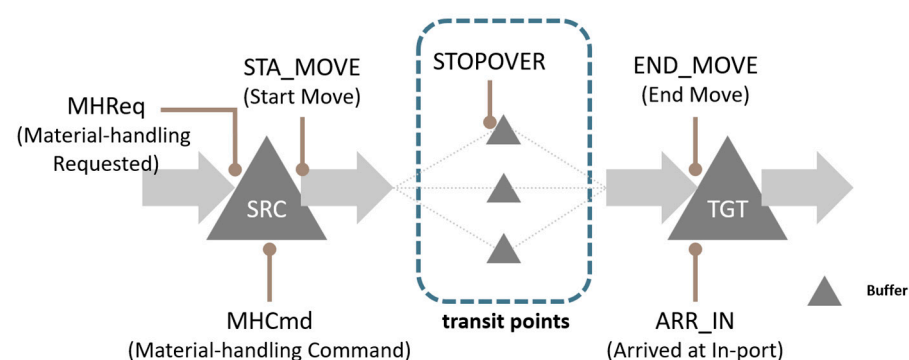


Figure 9. Reference model for the material-handling workflow.

The transfer of a WIP from a source (src) buffer to a target (tgt) buffer proceeds as follows. First, an MHReq (material-handling request) event is triggered to initiate the transfer. When a transport command is generated, an MHCmd (material-handling command) event is issued. As the WIP commences movement, an STA_MOVE (start move) event is triggered. Depending on the type of material-handling equipment, the MHCmd and STA_MOVE events may occur simultaneously (e.g., in conveyor systems) or experience a latency pending the arrival of the equipment (e.g., for AGVs or AMRs).

During transport, the WIP may pass through multiple transit points acting as intermediate buffers, where it may be temporarily stored. Upon arrival at the target buffer, the process concludes with the simultaneous triggering of END_MOVE (end move) and ARR_IN (arrived at input buffer) events. Table 5 provides the specifications of the events involved in this workflow.

Table 5. Discrete event model for the material-handling workflow.

Event		Description
MHReq	Triggering condition	Triggered by either (1) the selection of the transport WIP following LDReq, (2) the assignment of its destination following ULDRReq, or (3) WakeUp.
	Action	Find a material-handling equipment to transport the WIP from the source to the target. - <i>handler</i> = findMaterialHandler(<i>source</i>, <i>target</i>, <i>wip</i>) - if(<i>handler</i> != null) trigger MHCmd
MHCmd	Triggering condition	Triggered by MHReq, upon the assignment of a qualified material handler.
	Action	The transit of the material handler to the source location is initiated.
STA_MOVE	Triggering condition	After the MHCmd is issued, the material handler arrives at the source location, and the WIP becomes ready for transport.
	Action	The WIP transfer process is initiated.
STOPOVER	Triggering condition	After the MHCmd is issued, the WIP arrives at a transit point.
	Action	The next destination is identified, and the transfer process is resumed.
END_MOVE	Triggering condition	The WIP arrives at a target location.
	Action	The WIP transfer process is terminated. ARR_IN is triggered.
ARR_IN	Triggering condition	END_MOVE is triggered.
	Action	If the arrival location is a workstation, LDCom is triggered; if it is a buffer, ULDRReq is triggered.

4.3. Prototype Implementation

In this study, we designed and developed a prototype simulation testbed utilizing agent-based modeling techniques. This prototype serves as a digital-twin of the actual shop floor and is used to verify the operability and performance of the separately developed agent-based MESs.

4.3.1. Executors

Figure 10 depicts the behavioral model of each executor type as state-transition diagrams. These models are based on the two reference models proposed in Section 4.2, where state transitions are triggered by the specified events. Table 6 details the states shown in Figure 10 for each executor type, and Table 7 lists the events that trigger these transitions.

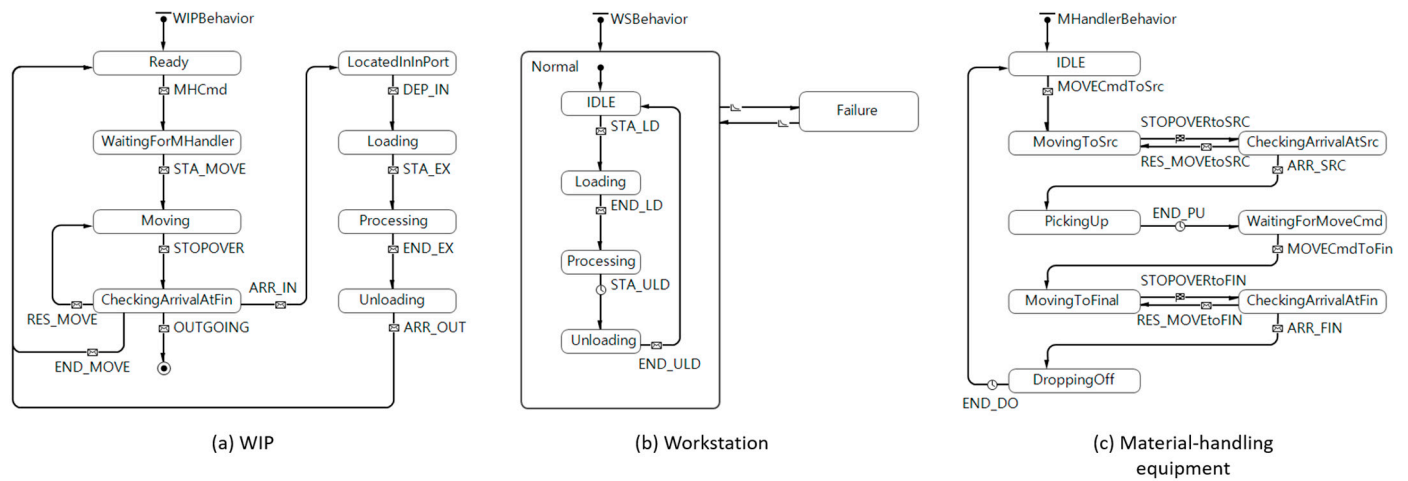


Figure 10. State-transition diagrams for individual executors, including the behavior of WIP-Tracker (a), the operational states of workstations (b), and the activities of material-handling equipment (c).

Table 6. State descriptions of individual executors.

Executor	State	Description
WIPTracker	Ready	A WIP is ready for transport.
	WaitingForMHandler	The WIP is waiting for the material-handling equipment to arrive.
	Moving	The WIP is on the move.
	CheckingArrivalAtFin	Check whether the WIP has reached its target workstation.
	LocatedInInPort	The WIP is located at the input port of its target workstation.
	Loading	The WIP is being loaded onto the workstation.
	Processing	The WIP is under processing at the workstation.
PEC	Unloading	The WIP is being unloaded from the workstation.
	IDLE	The workstation status: idle.
	Loading	The workstation is loading the WIP.
	Processing	The workstation is processing the WIP.
	Unloading	The workstation is unloading the WIP.
MHEC	Failure	The workstation status: failed.
	IDLE	The material-handling equipment status: idle
	MovingToSrc	The material-handling equipment is moving toward the source workstation where the WIP is located.
	CheckingArrivalAtSrc	Check if the material-handling equipment has arrived at the source workstation.
	PickingUp	The material-handling equipment is retrieving the WIP.
	WaitingForMoveCmd	The material-handling equipment is awaiting the move command.
	MovingToFin	The material-handling equipment is moving toward the target workstation of the WIP.
	CheckingArrivalAtFin	Check if the material-handling equipment has arrived at the target workstation.
	DroppingOff	The material-handling equipment is dropping off WIP.

Table 7. Description of triggering events for the state transition of individual executors.

Executor	Event	Description
WIPTracker	MHCmd	A material-handling command is received from WIPAgent.
	STA_MOVE	A WIP movement is initiated.
	STOPOVER	The WIP arrives at an intermediate station.
	RES_MOVE	The WIP movement resumes.
	END_MOVE	The WIP movement is completed.
	ARR_IN	The WIP arrives at the input port of a workstation.
	DEP_IN	The WIP has left the input port of a workstation.
	STA_EX	A WIP processing is initiated.
	END_EX	The WIP processing is completed.
	ARR_OUT	The WIP arrives at the output port of a workstation.
	OUTGOING	The WIP finishes processing and exits the system.
PEC	STA_LD	A WIP loading to the workstation is initiated.
	END_LD	The WIP loading to the workstation is completed.
	STA_ULD	A WIP unloading from the workstation is initiated.
	END_ULD	The WIP unloading from the workstation is completed.
MHEC	MOVECmdToSRC	A moving command to source station is received from MHAgent.
	STOPOVERtoSRC	The material-handling equipment arrives at the intermediate station during transit to the source station.
	RES_MOVEtoSRC	The material-handling equipment resumes transit to the source station.
	ARR_SRC	The material-handling equipment arrives at the source station.
	END_PU	The WIP pick-up is completed.
	MOVECmdToFIN	A move command to the target station is received from MHAgent.
	STOPOVERtoFIN	The material-handling equipment arrives at the intermediate station during the transit to the target station.
	RES_MOVEtoFIN	The material-handling equipment resumes transit to the target station.
	ARR_FIN	The material-handling equipment arrives at the target station.
	END_DO	The WIP drop-off is completed.

For example, if the “ARR_IN” event occurs while the WIPTracker is in the “CheckingArrivalAtFin” state, it transitions to the “LocatedInInPort” state. (The “ARR_IN” event is triggered when the MHEC notifies the WIPTracker upon arriving at the destination and unloading the WIP.) Subsequently, the WIPTracker notifies the PEC of the destination processing equipment that the WIP has arrived. If the PEC is in the “IDLE” state, the “STA_LD” event is triggered, causing it to transition to the “Loading” state. Simultaneously, the PEC begins feeding the WIP into the equipment; this action triggers the “DEP_IN” event in the WIPTracker, transitioning it to the “Loading” state. Upon the completion of the feeding process, the “STA_EX” and “END_LD” events occur in the WIPTracker and the PEC, respectively, transitioning both of them to their respective “Processing” states.

4.3.2. Agent–Executor Interaction

Figure 11 schematically illustrates the collaborative interactions between MES agents and shop-floor executors. By exchanging various messages, these collaborative objects trigger relevant events and state transitions in the recipient entities. Table 8 details the

messages exchanged between agents and executors, along with the resulting events. These events are categorized into three types—WIP events, processing events, and material-handling events—depending on the message sender.

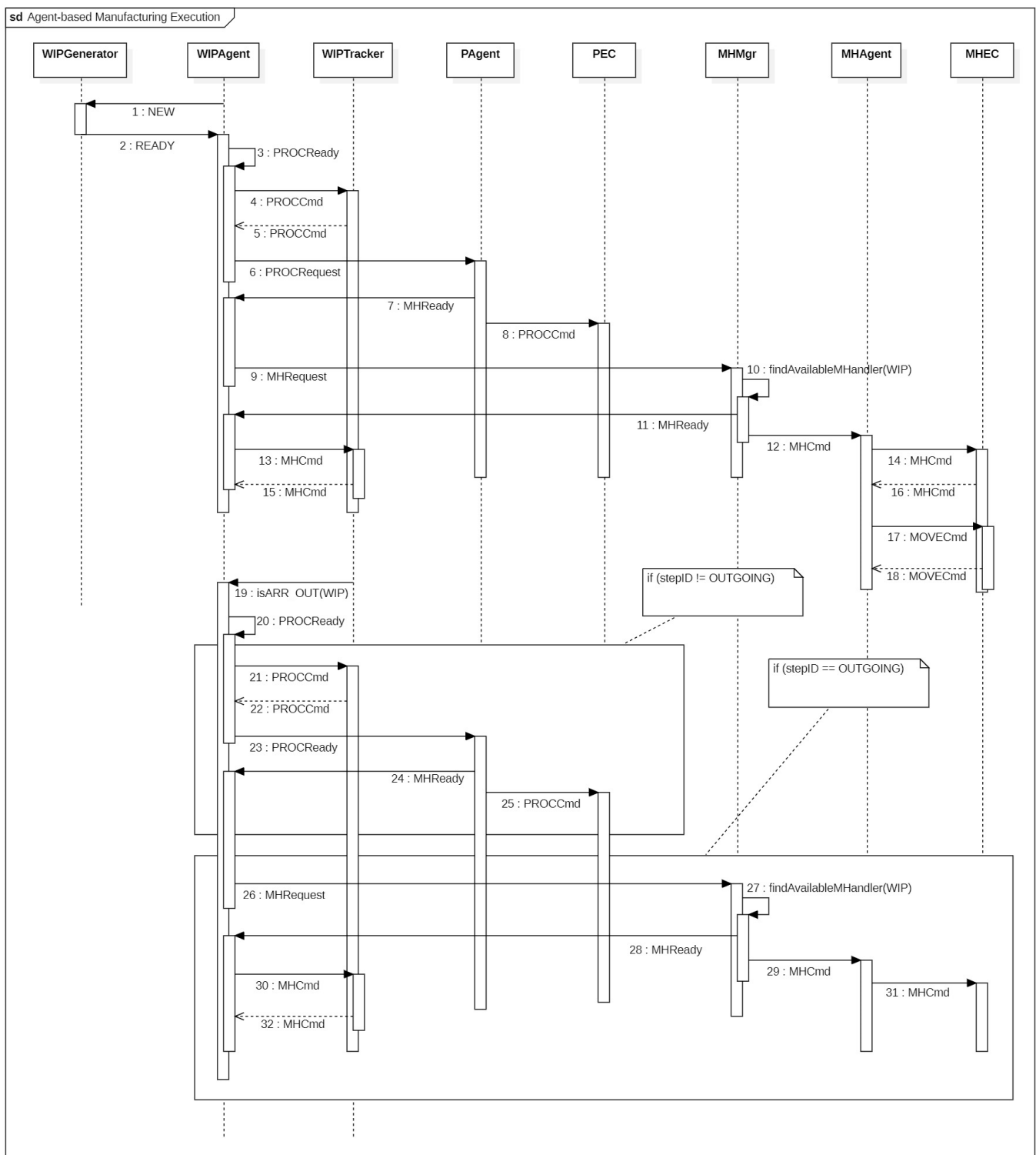


Figure 11. Manufacturing execution process based on collaboration among agents and executors.

Table 8. Event specifications and message structures exchanged between agents and executors.

Type	Sender Type	Receiver Type	Sender	Receiver	Performative	Event	Msg. Contents
WIP Event	Agent	Agent	WIPAgent	WIPGenerator	Request	NEW	[wipID ¹ productID ² stepID ³ curLocID ⁴]
				WIPAgent	Inform	PROCReady	[wipID stepID tgtID ⁵]
				PAgent	Inform	PROCRequest	[wipID stepID tgtID]
				MHMGr	Inform	MHRequest	[wipID srcID ⁶ tgtID handlerID ⁷]
		WIPGenerator	WIPAgent	Inform	READY	[wipID productID stepID curLocID]	
		Executor	WIPAgent	WIPTracker	Request	PROCCmd	[wipID stepID tgtID]
					Request	MHCmd	[wipID srcID tgtID handlerID]
					Inform	PROCCmd	[wipID stepID tgtID]
	Inform				MHCmd	[wipID srcID tgtID handlerID]	
	Executor	Agent	WIPTracker	WIPAgent	Inform	STA_MOVE	[wipID srcID tgtID handlerID fromID ⁸]
					Inform	STOPOVER	[wipID srcID tgtID handlerID toID ⁹]
					Inform	RES_MOVE	[wipID srcID tgtID handlerID fromID]
					Inform	END_MOVE	[wipID srcID tgtID handlerID toID]
					Inform	ARR_IN	[wipID tgtID portID ¹⁰]
					Inform	DEP_IN	[wipID tgtID portID]
					Inform	STA_EX	[wipID tgtID]
					Inform	END_EX	[wipID tgtID]
					Inform	ARR_OUT	[wipID tgtID portID]
					Inform	DEP_OUT	[wipID tgtID portID]
					Inform	OUTGOING	[wipID]
Processing Event					Agent	Agent	PAgent
	Executor	PAgent	PEC	Request		PROCCmd	[wipID stepID tgtID]
				Request		RESERVE_PORT	[wipID tgtID portID]
				Request		CANCEL_PORT	[wipID tgtID portID]
	Executor	Agent	PEC	PAgent	Inform	PROCCmd	[wipID stepID tgtID]
					Inform	STA_LD	[wipID tgtID portID]
					Inform	END_LD	[wipID tgtID portID]
					Inform	STA_EX	[wipID tgtID]
					Inform	END_EX	[wipID tgtID]
					Inform	STA_ULD	[wipID tgtID portID]
					Inform	END_ULD	[wipID tgtID portID]
					Inform	RESERVE_PORT	[wipID tgtID portID]
					Inform	CANCEL_PORT	[wipID tgtID portID]
					Inform	SEIZE_PORT	[wipID portID]
Inform	RELEASE_PORT	[wipID portID]					
Material-handling Event	Agent	Agent	MHMGr	WIPAgent	Inform	MHReady	[wipID srcID tgtID handlerID]
		Executor	MHAgent	MHAgent	Inform	MHCmd	[wipID srcID tgtID handlerID]
				MHEC	Request	MHCmd	[wipID srcID tgtID handlerID]
				MHEC	Request	MOVECmd	[wipID srcID tgtID fromID toID]
	Executor	Agent	MHEC	MHAgent	Inform	MHCmd	[wipID srcID tgtID handlerID]
					Inform	MOVECmd	[wipID srcID tgtID fromID toID]
					Inform	ARRIVE	[wipID srcID tgtID fromID toID]
					Inform	PICKUP	[wipID locID]
					Inform	DROPOFF	[wipID locID]
					Inform	RES_MOVEtoSRC	[wipID locID]
					Inform	ARR_SRC	[wipID locID]
					Inform	RES_MOVEtoFIN	[wipID locID]
Inform	ARR_FIN	[wipID locID]					

Table 8. Cont.

Type	Sender Type	Receiver Type	Sender	Receiver	Performative	Event	Msg. Contents
Executor Event	Executor	Executor	MHEC	WIPTracker	Inform	STA_MOVE	[wipID srcID tgtID handlerID fromID]
					Inform	STOPOVER	[wipID srcID tgtID handlerID toID]
					Inform	RES_MOVE	[wipID srcID tgtID handlerID toID]
					Inform	END_MOVE	[wipID srcID tgtID handlerID toID]
			PEC	WSPort	Request	RESERVED	[wipID tgtID portID]
					Request	CANCELED	[wipID tgtID portID]
					Request	SEIZED	[wipID portID]
					Request	RELEASED	[wipID portID]
			MHEC	WSPort	Inform	SEIZED	[wipID portID]
					Inform	RELEASED	[wipID portID]
			WSPort	PEC	Inform	SEIZE_PORT	[wipID portID]
					Inform	RELEASE_PORT	[wipID portID]
			PEC	WIPTracker	Inform	STA_LD	[wipID tgtID portID]
					Inform	ARR_IN	[wipID tgtID portID]
					Inform	DEP_IN	[wipID tgtID portID]
					Inform	STA_EX	[wipID tgtID]
					Inform	END_EX	[wipID tgtID]
					Inform	ARR_OUT	[wipID tgtID portID]
					Inform	DEP_OUT	[wipID tgtID portID]

¹ wipID—unique identifier of the WIP item. ² productID—identifier of the product type associated with the WIP. ³ stepID—identifier of the current processing step. ⁴ curLocID—identifier of the current location (e.g., workstation or transit point). ⁵ tgtID—identifier of the target workstation, where the current step is to be performed. ⁶ srcID—identifier of the source workstation, where the previous step was performed. ⁷ handlerID—identifier of the material-handling equipment assigned to the WIP. ⁸ fromID—identifier of the origin location for the current material-handling task. ⁹ toID—identifier of the destination location for the current material-handling task. ¹⁰ portID—identifier of the workstation port associated with the WIP.

Specifically, the WIPGenerator creates WIPTracker objects at the request of the WIPAgent. This implies that, upon the deployment of a physical WIP on the shop floor, a corresponding logical object is instantiated within the digital-twin. Furthermore, the MHMgr serves as a functional module managing the various material-handling equipment located on the shop floor. It maintains a list of MHAgents and provides an interface for identifying appropriate material-handling equipment.

The interaction sequence proceeds as follows: When the WIPAgent creates a WIPTracker via the WIPGenerator (triggering a “Ready” event), or when a WIP arrives at a destination and the station is ready for processing, a “PROCReady” event occurs. Subsequently, a “PROCCmd” message is sent to the associated WIPTracker to convey the processing command, and a “PROCRequest” message is transmitted to the PAgent responsible for the operation. Upon receipt, the PAgent sends an “MHReady” message to the WIPAgent, indicating readiness for transfer, and a “PROCCmd” message to its associated PEC.

Simultaneously, the WIPAgent sends an “MHRequest” to the MHMgr to request transfer. The MHMgr then searches for suitable material-handling equipment (findAvailableMHandler(WIP)). Once a suitable piece of material-handling equipment is selected, the MHMgr sends an “MHReady” message to the WIPAgent and an “MHCmd” message to the MHAgent associated with the selected material-handling equipment, thereby commanding the transport. The WIPAgent and MHAgent then relay the “MHCmd” message to their associated WIPTracker and MHEC, respectively.

When the WIPTracker detects that the WIP has been processed and placed at the output port, it transmits an “ARR_OUT” message to the WIPAgent. If remaining operations exist (i.e., stepID != OUTGOING), then the WIPAgent identifies the next piece of processing

equipment and repeats the aforementioned process. Conversely, if no further operations remain (i.e., $\text{stepID} == \text{OUTGOING}$), then the system initiates transport to the outgoing location, terminating the collaboration process.

5. Simulation Experiments

5.1. Experimental Configuration

To demonstrate the effectiveness of the agent-based decentralized manufacturing execution mechanism proposed in this study, we conducted a comparative simulation study. For a typical task allocation scenario, we compared the proposed EmNet-based decision-making method with a centralized problem-solving approach and a greedy distributed approach.

5.1.1. Comparative Policies

The centralized policy aims to derive an optimal (or near-optimal) global schedule by aggregating information on all tasks and resources at a single point. In this implementation, ready tasks are first categorized by capability; then, tasks within the same category are grouped by due date to form batches, ensuring that each batch adheres to capacity constraints. Each batch is then assigned to the resource offering the earliest completion time. Conversely, in the greedy distributed approach, each task is immediately assigned—using a greedy rule—to the resource that can complete it the earliest. Considering the batch and WIP cost structure of production sites, greedy distribution serves as a suitable baseline for comparison, though its potential for performance optimization is inherently limited.

In the proposed method, each employer agent evaluates bids using Equation (7):

$$GA_i = 0.5 \cdot \mu_1(t_i) + 0.5 \cdot \mu_2(c_i) \quad (7)$$

where t_i and c_i denote the expected tardiness and the cost associated with task i , respectively. Especially, the cost is calculated using Equation (8):

$$c_i = 0.5 \cdot s_i + 30.0 \cdot (1 - b_i) + 0.1 \cdot u_i \quad (8)$$

where s_i and b_i represents the necessity of a setup (or a binary setup indicator) and the fill rate of the batch associated with task i , respectively; u_i denotes the utilization rate of the bidder. The weights assigned to each term were determined empirically. In addition, each membership degree is calculated using Equation (9):

$$\mu_i(x_i) = 1 - \frac{1}{1 + e^{-x_i}} \quad (9)$$

5.1.2. Experimental Scenario

The basic experimental setup and environment follow that of Shin [26]. Two homogeneous resources (R_1 and R_2) were used, with a batch capacity of 20 and a setup time of 10. Each task had a uniformly distributed required capability (1–20), quantity (3–10), and due date (500–2000). The processing time followed a normal distribution with a mean of 100 and a standard deviation of 10, truncated at 70–130. The batch processing time, which grouped tasks of the same capability, was reflected as the maximum processing time of the tasks in the batch. The release time of each task was distributed within the range of 0–300 to generate load fluctuations. The total number of tasks was set to 100 and 1000 for the small-scale and large-scale scenarios, respectively.

The experimental design comprised two distinct cases: a nominal case, where simple repetitive experiments were performed under standard settings, and a disturbance case,

where various disturbances occurred sequentially. The perturbation events included the following: (1) random cancellation of 5% of orders at $t = 400$, (2) insertion of 30 rush jobs at $t = 500$, (3) 120 min failure of resource R_1 at $t = 600$, (4) reduction of batch capacity to 15 at $t = 700$, (5) temporary 200 min failure of resource R_2 's capability 3 at $t = 800$, (6) a setup time increase of 5 at $t = 900$, and (7) a 10% increase in processing time at $t = 1000$. For a fair comparison, the same replanning principle was applied at each perturbation event, which rescheduled jobs that were not started according to the corresponding policy rule. The number of experiments was 30 in both cases.

5.2. Results

In the nominal case with 100 tasks, the average latency was 155 min for the centralized approach, 1420 min for the proposed EmNet-based method, and 1550 min for the greedy approach (see Table 9). In this small-scale deterministic scenario, the centralized approach demonstrated overwhelming dominance across most metrics, with the exception of the number of setups. Meanwhile, the proposed method showed only a marginal improvement over the greedy distributed approach. On the other hand, in the nominal case with 1000 tasks, average latency followed the order presented here: centralized (24,234 min) < proposed (24,257 min) < greedy (26,175 min) (see Table 10). The average latency of the proposed and centralized methods differed by only 0.09%, and both outperformed the greedy distributed approach by about 7%. The proposed method resulted in a slightly lower number of batches (921.8) compared to the centralized approach (924.1), and setup times were similar for both policies. As the scale increases, the local placement and setup optimization of the proposed method converges to the global optimum, and the gap with the simple distributed (greedy) approach becomes clearer.

Table 9. Simulation results (nominal case with 100 tasks and 30 repetitions).

Mean	Greedy (Distributed)	Centralized	Proposed
Tardiness (95% CI)	1549.54 ± 10.39	154.68 ± 8.59	1419.51 ± 22.91
Tardiness rate (95% CI)	88.24% ± 0.28	50.51% ± 1.22	76.01% ± 0.62
Number of batches	100.00	44.27	44.57
Number of setups	927.00	416.60	188.90
Number of messages	0.00	0.00	500.00

Table 10. Simulation results (nominal case with 1000 tasks and 30 repetitions).

Mean	Greedy (Distributed)	Centralized	Proposed
Tardiness (95% CI)	26,175.37 ± 37.61	24,234.48 ± 81.13	24,256.88 ± 95.04
Tardiness rate (95% CI)	97.79% ± 0.09	97.60% ± 0.10	97.61% ± 0.09
Number of batches	1000.00	924.13	921.77
Number of setups	9445.00	8919.67	9093.67
Number of messages	0.00	0.00	5000.00

In the disturbance case with 1000 tasks, average latency followed the order presented here: proposed (221,910 min) < centralized (232,693 min) < greedy (415,806 min) (see Table 11). The proposed method achieved approximately 4.63% lower average latency than the centralized approach and approximately 46.6% lower than the greedy one. A one-tailed t -test assuming unequal variances (Welch's t -test) confirmed that the proposed method yields significantly lower latency compared to the centralized approach ($p < 0.001$). Notably, the total number of setups was reduced by approximately 95.7%, dropping to 359.7 (proposed) from 8376.5 (centralized). This demonstrates that partial renegotiation significantly lowers reconfiguration costs in repetitive rescheduling scenarios. The number

of batches was virtually identical between the two policies. Figure 12 shows the simulation results of 30 repetitions. The raw data for these simulation experiments are provided in the Supplementary Materials (Table S1).

Table 11. Simulation results (disturbance case with 1000 tasks and 30 repetitions).

Mean	Greedy (Distributed)	Centralized	Proposed
Tardiness (95% CI)	415,805.96 $\pm$ 483.76	232,692.70 $\pm$ 937.56	221,909.89 $\pm$ 1706.45
Tardiness rate (95% CI)	99.22% $\pm$ 0.01	99.15% $\pm$ 0.03	99.14% $\pm$ 0.02
Number of batches	1000.00	583.00	585.83
Number of setups	14,512.33	8376.50	359.67
Number of messages	0.00	0.00	40,387.00

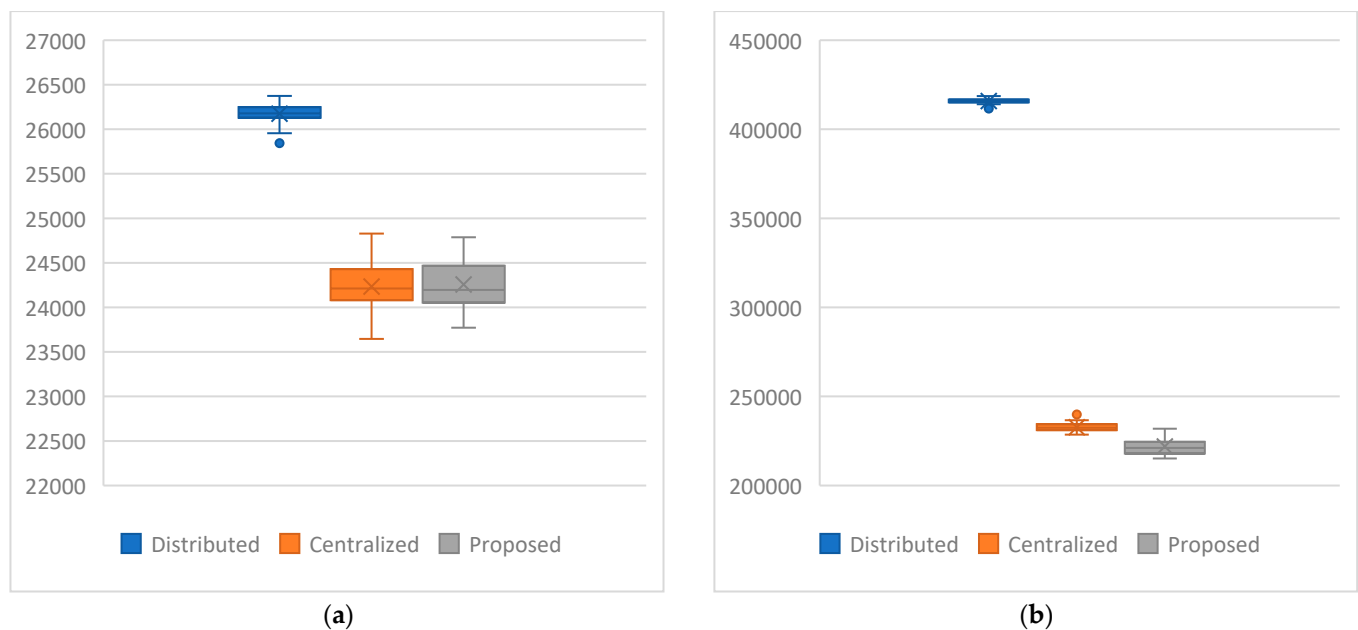


Figure 12. Comparison of mean tardiness (minutes): (a) tardiness in nominal case with 1000 tasks and 30 repetitions; (b) tardiness in disturbance case with 1000 tasks and 30 repetitions.

5.3. Discussion

While the centralized approach minimizes latency in static environments, the proposed method achieves comparable operational efficiency while incurring significantly lower reconfiguration costs. Furthermore, in dynamic environments, the proposed architecture demonstrates superior resilience, outperforming the centralized approach in terms of both latency and operational stability.

Considering setup switching costs and the importance of batch configuration, the centralized approach—employing global bundling and balanced assignment—is advantageous for reducing average delays and minimizing unnecessary setups. However, it necessitates complete, real-time information sharing and frequent recalculation among the production resources assigned to tasks. This limitation imposes a significant re-planning burdens in the event of major environmental changes. Although this centralized decision-making offers system-wide optimization, it may not adequately reflect the distributed and autonomous nature of real-world manufacturing environments.

Conversely, while the greedy distributed approach benefits from minimal implementation and communication overheads, it tends to fragment batches, as tasks requiring the same capability are randomly distributed across production resources. This leads to increased batch counts and frequent setup changes, thereby reducing overall flow efficiency

and potentially increasing average delays. This phenomenon is characteristic of competitive allocation scenarios where information sharing among decision-makers is limited.

Ultimately, in environments characterized by frequent disruptions and changes, the proposed method's partial and incremental re-contracting mechanism offers distinct advantages over the centralized policy, which relies heavily on global re-computation. These advantages are particularly evident in terms of resilience and operational costs, aligning with the theoretical expectations of a collaborative model based on supplier-centric capacity allocation and limited, data-space-based information sharing. Furthermore, the proposed method achieves performance comparable to the centralized approach in large-scale static environments. This result is attributed to the decision-making framework grounded in a quasi-hierarchical structure established among production resources via EmNet. Fundamentally, while individual resources make decisions independently and autonomously, the employment relationship enables employer units to coordinate their actions, thereby enhancing overall performance.

6. Conclusions

This study proposes an agent-based distributed, collaborative MES architecture that was designed to overcome the structural rigidity of conventional centralized systems, thereby enhancing responsiveness and reconfigurability. By adopting the EmNet-based collaboration model, the proposed system enables dynamic organization and collaboration among distributed manufacturing resources. The architecture models the manufacturing environment through three primary agents—WIPagents, PAgents, and MHAgents—which collaborate via a data-space-based platform to execute relation-forming and task-allocating processes. To validate the feasibility and operability of the proposed architecture, a digital-twin-based simulation testbed was developed. This testbed integrates the agent-based decision-making layer with a discrete event-based execution layer, successfully emulating the interaction between MES logic and shop-floor executors.

Simulation experiments were conducted to compare the proposed method against a centralized approach and a simple distributed (greedy) approach. The experiments were conducted under two conditions: a nominal stable environment and a disturbed environment subject to equipment failures, rush orders, and capacity fluctuations. In nominal environments, the proposed method achieved a performance comparable to that of the centralized global optimization strategy, with only a marginal difference in average latency. However, in environments subject to frequent disturbances, the proposed architecture demonstrated superior resilience. Notably, it significantly reduced the number of setups and achieved lower average latency compared with both the centralized and greedy approaches. These results confirm that EmNet's partial and incremental re-contracting mechanism minimizes reconfiguration costs and enhances system agility.

However, this study is limited by the absence of specific decision-making models and the reliance on hypothetical scenarios for performance verification. Consequently, the scope of validation was restricted to demonstrating the operability and conceptual advantages of the proposed mechanism. Furthermore, it was difficult to conduct quantitative comparisons with various existing multi-agent-based models. As a result, the benefits of the proposed architecture could not be fully demonstrated in this study. Moreover, this study did not explicitly address the communication overheads associated with multi-layered collaboration among autonomous and distributed decision makers.

Future research will focus on integrating sophisticated problem-solving algorithms into the proposed collaboration model. Through this integration, we aim to establish a foundation for quantitatively evaluating the superiority of the architecture. Specifically, by integrating reinforcement learning algorithms into the negotiation process—including

partner selection, bid generation, and evaluation—we aim to enhance the decision-making intelligence of individual agents and ultimately improve global optimality. Furthermore, we plan to validate this enhanced logic through empirical experiments in an actual manufacturing environment. These efforts will serve to verify the fidelity of the digital-twin and demonstrate the operability and performance of the proposed system architecture, as well as its overall utility. Moreover, we intend to investigate the communication overheads—specifically those arising from the vertical propagation of job and task contracts within the multi-layered hierarchical structure of production resources in actual industry environments. This will allow us to evaluate the computation cost and scalability of the proposed architecture. In addition, further research is required on the technical details of the collaboration platform—such as ontologies, semantics, and information models—to support the communication and cooperative decision-making of autonomous agents. These advancements are expected to facilitate the practical implementation of the architecture proposed in this study.

Supplementary Materials: The following supporting information can be downloaded at: <https://www.mdpi.com/article/10.3390/app16010386/s1>, Table S1: raw data of simulation results.

Funding: This work was supported by a project for Smart Manufacturing Innovation R&D funded by the Korea Ministry of SMEs and Startups in 2025 (Project No. RS-2022-00140968).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data presented in this study are available in the article.

Conflicts of Interest: The author declares no conflicts of interest.

References

- Cheng, Y.; Zhang, Y.P.; Tao, F.; Nee, A.Y.C. Cyber–physical integration for moving digital factories forward towards smart manufacturing: A survey. *Int. J. Adv. Manuf. Technol.* **2018**, *97*, 1209–1221. [\[CrossRef\]](#)
- Zheng, P.; Wang, H.; Sang, Z.; Zhong, R.Y.; Liu, Y.; Liu, C.; Mubarak, K.; Yu, S.; Xu, X. Smart manufacturing systems for Industry 4.0: Conceptual framework, scenarios, and future perspectives. *Front. Mech. Eng.* **2018**, *13*, 137–150. [\[CrossRef\]](#)
- Tao, F.; Qi, Q.; Wang, L.; Nee, A.Y.C. Digital twins and cyber–physical systems toward smart manufacturing and Industry 4.0: Correlation and comparison. *Engineering* **2019**, *5*, 653–661. [\[CrossRef\]](#)
- He, B.; Bai, K.J. Digital twin-based sustainable intelligent manufacturing: A review. *Adv. Manuf.* **2021**, *9*, 1–21. [\[CrossRef\]](#)
- Valckenaers, P. Perspective on holonic manufacturing systems: PROSA becomes ARTI. *Comput. Ind.* **2020**, *120*, 103226. [\[CrossRef\]](#)
- Jaskó, S.; Skrop, A.; Holczinger, T.; Chován, T.; Abonyi, J. Development of manufacturing execution systems in accordance with Industry 4.0 requirements: A review of standard- and ontology-based methodologies and tools. *Comput. Ind.* **2020**, *123*, 103300. [\[CrossRef\]](#)
- Zwolińska, B.; Tubis, A.A.; Chamier-Gliszczyński, N.; Kostrzewski, M. Personalization of the MES system to the needs of highly variable production. *Sensors* **2020**, *20*, 6484. [\[CrossRef\]](#) [\[PubMed\]](#)
- Boccella, A.R.; Centobelli, P.; Cerchione, R.; Murino, T.; Riedel, R. Evaluating centralized and heterarchical control of smart manufacturing systems in the era of Industry 4.0. *Appl. Sci.* **2020**, *10*, 755. [\[CrossRef\]](#)
- Shen, W.; Hao, Q.; Yoon, H.J.; Norrie, D.H. Applications of agent-based systems in intelligent manufacturing: An updated review. *Adv. Eng. Inform.* **2006**, *20*, 415–431. [\[CrossRef\]](#)
- Leitão, P.; Karnouskos, S.; Ribeiro, L.; Lee, J.; Strasser, T.; Colombo, A.W. Smart agents in industrial cyber–physical systems. *Proc. IEEE* **2016**, *104*, 1086–1101. [\[CrossRef\]](#)
- Tang, H.; Li, D.; Wang, S.; Dong, Z. CASOA: An architecture for agent-based manufacturing system in the context of Industry 4.0. *IEEE Access* **2018**, *6*, 12746–12754. [\[CrossRef\]](#)
- Kovalenko, I.; Tilbury, D.; Barton, K. The model-based product agent: A control-oriented architecture for intelligent products in multi-agent manufacturing systems. *Control Eng. Pract.* **2019**, *86*, 105–117. [\[CrossRef\]](#)
- Blanc, P.; Demongodin, I.; Castagna, P. A holonic approach for manufacturing execution system design: An industrial application. *Eng. Appl. Artif. Intell.* **2008**, *21*, 315–330. [\[CrossRef\]](#)

14. Fang, J.; Huang, G.Q.; Li, Z. Event-driven multi-agent ubiquitous manufacturing execution platform for shop floor work-in-progress management. *Int. J. Prod. Res.* **2013**, *51*, 1168–1185. [\[CrossRef\]](#)
15. Monostori, L.; Valckenaers, P.; Dolgui, A.; Panetto, H.; Brdys, M.; Csáji, B.C. Cooperative control in production and logistics. *Annu. Rev. Control* **2015**, *39*, 12–29. [\[CrossRef\]](#)
16. Cupek, R.; Ziebinski, A.; Huczala, L.; Erdogan, H. Agent-based manufacturing execution systems for short-series production scheduling. *Comput. Ind.* **2016**, *82*, 245–258. [\[CrossRef\]](#)
17. Vyskočil, J.; Douda, P.; Novák, P.; Wally, B. A digital twin-based distributed manufacturing execution system for Industry 4.0 with AI-powered on-the-fly replanning capabilities. *Sustainability* **2023**, *15*, 6251. [\[CrossRef\]](#)
18. Leitão, P. Agent-based distributed manufacturing control: A state-of-the-art survey. *Eng. Appl. Artif. Intell.* **2009**, *22*, 979–991. [\[CrossRef\]](#)
19. Novas, J.M.; Van Belle, J.; Saint Germain, B.; Valckenaers, P. A collaborative framework between a scheduling system and a holonic manufacturing execution system. In *Service Orientation in Holonic and Multi Agent Manufacturing and Robotics*; Springer: Berlin/Heidelberg, Germany, 2013; pp. 3–17.
20. Pulikottil, T.; Estrada-Jimenez, L.A.; Rehman, H.U.; Mo, F.; Nikghadam Hojjati, S.; Barata, J. Agent-based manufacturing—Review and expert evaluation. *Int. J. Adv. Manuf. Technol.* **2023**, *127*, 2151–2180. [\[CrossRef\]](#)
21. Barbosa, J.; Leitão, P.; Adam, E.; Trentesaux, D. Dynamic self-organization in holonic multi-agent manufacturing systems: The ADACOR evolution. *Comput. Ind.* **2015**, *66*, 99–111. [\[CrossRef\]](#)
22. Leitão, P.; Restivo, F. ADACOR: A holonic architecture for agile and adaptive manufacturing control. *Comput. Ind.* **2006**, *57*, 121–130. [\[CrossRef\]](#)
23. Panetto, H.; Molina, A. Enterprise integration and interoperability in manufacturing systems: Trends and issues. *Comput. Ind.* **2008**, *59*, 641–646. [\[CrossRef\]](#)
24. Iarovyi, S.; Mohammed, W.M.; Lobov, A.; Ferrer, B.R.; Lastra, J.L.M. Cyber-physical systems for open-knowledge-driven manufacturing execution systems. *Proc. IEEE* **2016**, *104*, 1142–1154. [\[CrossRef\]](#)
25. Shin, M. Employment contract-based self-organizing mechanism for production resources. *J. Korean Inst. Ind. Eng.* **2020**, *46*, 282–295.
26. Shin, M. Development of collaboration model for data space-based open collaboration platform in continuous process industries. *Sustainability* **2025**, *17*, 126.
27. Rolón, M.; Martínez, E. Agent-based modeling and simulation of an autonomic manufacturing execution system. *Comput. Ind.* **2012**, *63*, 53–78. [\[CrossRef\]](#)
28. Van Brussel, H.; Wyns, J.; Valckenaers, P.; Bongaerts, L.; Peeters, P. Reference architecture for holonic manufacturing systems: PROSA. *Comput. Ind.* **1998**, *37*, 255–274. [\[CrossRef\]](#)
29. Babiceanu, R.F.; Chen, F.F. Development and applications of holonic manufacturing systems: A survey. *J. Intell. Manuf.* **2006**, *17*, 111–131. [\[CrossRef\]](#)
30. Valckenaers, P.; Van Brussel, H.; Verstraete, P.; Saint Germain, B. Schedule execution in autonomic manufacturing execution systems. *J. Manuf. Syst.* **2007**, *26*, 75–84. [\[CrossRef\]](#)
31. Warnecke, H.-J. *The Fractal Company: A Revolution in Corporate Culture*; Springer: Berlin/Heidelberg, Germany, 1993.
32. Ryu, K.; Jung, M. Agent-based fractal architecture and modelling for developing distributed manufacturing systems. *Int. J. Prod. Res.* **2003**, *41*, 4233–4255. [\[CrossRef\]](#)
33. Shin, M.; Mun, J.; Lee, K.; Jung, M. r-FrMS: A relation-driven fractal organisation for distributed manufacturing systems. *Int. J. Prod. Res.* **2009**, *47*, 1791–1814.
34. Ueda, K. A concept for bionic manufacturing systems based on DNA-type information. In *Human Aspects in Computer Integrated Manufacturing*; Elsevier: Amsterdam, The Netherlands, 1992; pp. 853–863.
35. Tharumarajah, A. Comparison of the bionic, fractal and holonic manufacturing system concepts. *Int. J. Comput. Integr. Manuf.* **1996**, *9*, 217–226. [\[CrossRef\]](#)
36. Smith, R.G. The contract net protocol: High-level communication and control in a distributed problem solver. *IEEE Trans. Comput.* **1980**, *29*, 1104–1113. [\[CrossRef\]](#)
37. Baker, A.D. A survey of factory control algorithms that can be implemented in a multi-agent heterarchy: Dispatching, scheduling, and pull. *J. Manuf. Syst.* **1998**, *17*, 297–320. [\[CrossRef\]](#)
38. Ryu, K.; Yücesan, E.; Jung, M. Dynamic restructuring process for self-reconfiguration in the fractal manufacturing system. *Int. J. Prod. Res.* **2006**, *44*, 3105–3129. [\[CrossRef\]](#)
39. Parunak, H.V.D. Industrial and practical applications of distributed artificial intelligence. In *Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence*; MIT Press: Cambridge, MA, USA, 1999; pp. 377–421.
40. Hussain, M.S.; Ali, M. A multi-agent-based dynamic scheduling of flexible manufacturing systems. *Glob. J. Flex. Syst. Manag.* **2019**, *20*, 267–290. [\[CrossRef\]](#)
41. Arai, T.; Aiyama, Y.; Sugi, M.; Ota, J. Holonic assembly system with Plug and Produce. *Comput. Ind.* **2001**, *46*, 289–299. [\[CrossRef\]](#)

42. Shin, M.; Ryu, K. Agent-based resource model for relation-driven fractal organization. *ICIC Expr. Lett.* **2013**, *7*, 1539–1544.
43. Shin, M. Employment contract-based management model of production resources on relation-driven fractal organization. *J. Korean Inst. Ind. Eng.* **2013**, *39*, 278–289.
44. Nagel, L.; Lycklama, D. Design Principles for Data Spaces. 2021. Available online: <https://design-principles-for-data-spaces.org/> (accessed on 28 October 2024).
45. Gaia-X European Association for Data and Cloud AISBL. GAIA-X Architecture Document, 22.04 Release. Available online: <https://gaia-x.eu/wp-content/uploads/2022/06/Gaia-x-Architecture-Document-22.04-Release.pdf> (accessed on 28 October 2024).
46. Schöppenthau, F.; Patzer, F.; Schnebel, B.; Watson, K.; Baryshnikov, N.; Obst, B.; Chauhan, Y.; Kaefer, D.; Usländer, T.; Kulkarni, P. Building a digital manufacturing as a service ecosystem for Catena-X. *Sensors* **2023**, *23*, 7396. [CrossRef]
47. Usländer, T.; Baumann, M.; Boschert, S.; Rosen, R.; Sauer, O.; Stojanovic, L.; Wehrstedt, J.C. Symbiotic evolution of digital twin systems and dataspaces. *Automation* **2022**, *3*, 378–399. [CrossRef]
48. MESA International. History of the MESA Models. Available online: https://mesa.org/topics-resources/mesa-model/history-of-the-mesa-models/?utm_source=chatgpt.com (accessed on 20 November 2025).
49. International Society of Automation (ISA). ANSI/ISA-95.00.03-2013 (IEC 62264-3 Modified), Enterprise-Control System Integration—Part 3: Activity Models of Manufacturing Operations Management. Available online: <https://www.isa.org/products/ansi-isa-95-00-03-2013-enterprise-control-system-i> (accessed on 20 November 2025).

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.